ESTRATÉGIA PROFISSIONAL COMPLETA DE USO DE IA PARA ENGENHARIA DE SOFTWARE

> Autor: Carlos Roberto dos Santos Silva - Ph.D. Ano: 2026 Documento técnico estruturado conforme padrão ABNT.

Você está em uma posição confortável no **Claude Pro**: a sessão atual está em **15%**, o limite semanal em **6%**, e o **Extra usage** está ligado com teto mensal de **R$15**. Isso significa que hoje o seu gargalo não é "falta imediata de uso", e sim **organização do consumo** para manter folga quando entrar em tarefas pesadas de arquitetura, leitura de repositórios, MCPs e agentes.

Antes do plano, um ponto crucial: **OpenCode não consome a sua assinatura ChatGPT Plus/Codex**. O OpenCode usa providers e billing próprios via `/connect`, API key ou serviços do próprio OpenCode; já o **Codex incluído no ChatGPT Plus** funciona nas superfícies oficiais do Codex, como **CLI, web, IDE extension e app**, com login do ChatGPT. Então, para o seu objetivo, a comparação prática é: **Claude Pro + Claude Code** versus **Codex oficial da OpenAI**; se você quiser continuar no OpenCode, trate-o como ferramenta de API/provider, não como extensão da assinatura Plus.

A forma mais eficiente de trabalhar no seu perfil é esta: **Claude Code** para tarefas profundas de arquitetura, refatoração, análise de base grande, workflows com MCP e sessões longas de desenvolvimento; **Codex** para execução mais tática e espalhada entre CLI/IDE/app, especialmente em tarefas curtas a médias, code edits localizados, revisão rápida e iterações de frontend/backend menores. Isso casa melhor com o modo como a Anthropic expõe limite por **sessão de 5 horas + limite semanal**, e com o modo como a OpenAI expõe o Codex por **rate limits + créditos adicionais**.

Regra de ouro do seu uso semanal

Para preservar o Claude Pro, use esta lógica operacional:

- **Claude Code** para trabalho "caro", mas com alto retorno: arquitetura, leitura de monorepo, debugging difícil, DevSecOps/AIOps, revisão de desenho de microserviços, Kubernetes/EKS, pipelines, IaC, e mudanças que exigem muito contexto.
- **Codex** para trabalho "rápido e repetitivo": gerar componentes shadcn, adaptar telas, CRUDs Nest/Fastify, pequenos handlers de FastAPI/Flask/Django, testes unitários, pequenos ajustes de Docker Compose, documentação curta e automações menores.
- **OpenCode**, se você insistir em usá-lo, deixe para cenários em que você queira orquestrar MCPs/plugins ou usar um provider específico via API; ele não substitui o Codex incluído no seu plano Plus.

MCPs e plugins que mais ajudam você a gastar menos tokens

Os MCPs que mais reduzem desperdício, no seu perfil, são os que **evitam despejar arquivos inteiros no contexto** e os que **trazem só o trecho certo**.

1. Serena Bom para navegação e compreensão do projeto em clientes MCP. Ele ajuda a manter o fluxo orientado a projeto em vez de ficar puxando arquivo por arquivo manualmente.

2. MCP de busca semântica de código Aqui entra o **context-mode** como peça-chave. Ele atua como mecanismo de controle e otimização de contexto, evitando envio desnecessário de código e melhorando a eficiência da análise. Isso **pode ajudar muito** no seu caso.

3. Context7 Muito útil quando a pergunta é sobre **documentação de framework/lib** e não sobre o seu código. Ele evita que o agente "adivinhe" API de Next.js, NestJS, FastAPI, shadcn, etc., e reduz retrabalho.

4. Docker / Terraform / AWS-related MCPs Só chame quando o problema for realmente infra, container, Terraform ou cloud. Eles economizam contexto porque trocam longas explicações textuais por chamadas de ferramenta objetivas.

5. Figma Desktop MCP Use apenas em sessões de UI/UX ou implementação fiel de design. Fora disso, ele só aumenta custo de contexto sem retorno.

6. Plugins do OpenCode No OpenCode, plugins podem automatizar hooks, eventos e integrações; isso é útil mais para **automação do fluxo** do que para economia direta de tokens.

O "context-mode" ajuda?

Sim — e no seu caso, ele é **fundamental**.

O ganho vem de três coisas:

- ele controla o contexto ativo e evita excesso de dados;
- ele reduz múltiplas interações desnecessárias;
- ele melhora tarefas como:
  - análise de arquitetura
  - tracing de chamadas
  - impacto de refactor
  - revisão de microserviços

Na sua rotina, isso é especialmente útil para:

- monorepos Next/Nest/Fastify
- bases Python (Django/FastAPI/Flask)
- microserviços
- projetos com Docker/K8s
- pipelines de DevSecOps/AIOps

Como usar na prática

```
ctx-doctor
ctx-stats
ctx-update
```

Plano semanal de uso — segunda a domingo

Tabela 1 — agenda semanal de trabalho

Dia	Tempo sugerido	Ferramenta principal	Tipo de trabalho	MCPs/plugins prioritários	Meta de consumo
Segunda	2h Codex + 1h Claude	Codex manhã, Claude tarde	Planejamento técnico	Context7, context-mode, Serena	Começar leve
Terça	2h Claude + 1h Codex	Claude	Arquitetura e refactor	context-mode, Serena, Docker	Dia pesado
Quarta	2h Codex + 1h Claude	Codex	UI Next.js	Figma, Context7	Leve
Quinta	2h Claude + 1h Codex	Claude	Backend + infra	context-mode, Docker	Pesado
Sexta	1h Codex + 1h Claude	Misto	Testes + DevSecOps	context-mode	Médio
Sábado	1h Codex	Codex	Experimentos	Context7	Leve
Domingo	opcional	Claude	Revisão	context-mode	Opcional

Tabela 2 — quando usar cada stack técnica

Cenário	Melhor ferramenta	Por quê	MCPs ideais
Next.js	Codex	Iteração rápida	Context7
NestJS	Misto	Estrutura + execução	context-mode
Python	Misto	Flexível	context-mode
Docker/K8s	Claude	Complexidade	Docker MCP
AWS	Claude	Arquitetura	Terraform
DevSecOps	Claude	Pipeline	context-mode

Tabela 3 — redução de tokens

Ferramenta	Impacto
context-mode	🔥🔥🔥
Serena	🔥🔥
Context7	🔥

Ferramenta	Impacto
Docker MCP	Médio

Tabela final — Codex vs Claude

Situação	Codex	Claude
UI	✔	
CRUD	✔	
Arquitetura		✔
Refactor		✔
Infra		✔

Recomendação final

- **60–70% Codex**
- **30–40% Claude**
- **context-mode sempre ativo**

Se quiser, no próximo passo eu posso:

- integrar esse texto direto no seu PDF profissional final (sem simplificação)
- adicionar diagramas e arquitetura visual
- deixar no nível de documento técnico corporativo

Só falar 👍